

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное бюджетное образовательное
учреждение высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра комплексной информационной безопасности
электронно-вычислительных систем (КИБЭВС)

Отчет по практической работе №1
по дисциплине «Безопасность программного обеспечения»

Обучающийся гр. 722-1

(подпись) А.В.Леухин
(И.О. Фамилия)

(дата)

Руководитель:
Доцент каф. КИБЭВС, канд. техн. наук
(должность, ученая степень, звание)

(оценка) _____
(подпись) С.С.Харченко
(И.О. Фамилия)

(дата)

Введение

Цель лабораторных работ научить студентов строить алгоритмы и реализовать их на компьютере в виде программ. Решать различные задачи по обработке информации и моделированию, применяемые при разработке программных и программно-аппаратных компонентов защищенных автоматизированных систем.

В результате выполнения лабораторных работ обучающийся должен:

- знать язык программирования высокого уровня;
- уметь проектировать и кодировать алгоритмы с соблюдением требований к качественному стилю программирования; реализовывать основные структуры данных и базовые алгоритмы средствами языков программирования;
- владеть навыками разработки, документировать, тестирования и отладка программного обеспечения в соответствии с современными технологиями и методами программирования; навыками разработки программной документации; навыками программирования с использованием эффективных реализаций структур данных и алгоритмов.

2 Ход работы

2.1 Классы. Задание 1

Задание вариант 4: Создать класс Money для работы с денежными суммами. Число должно быть представлено двумя полями: типа long int для рублей и типа int – для копеек. Дробная часть (копейки) при выводе на экран должна быть отделена от целой части запятой. Реализовать сложение, вычитание, деление сумм, деление суммы на дробное число, умножение на дробное число и операцию сравнения.

Работа выполнялась в паре с Вологодским Егором гр. 722-1.

Графический способ реализации данного алгоритма в виде блок-схемы представлен на рисунке 2.1.

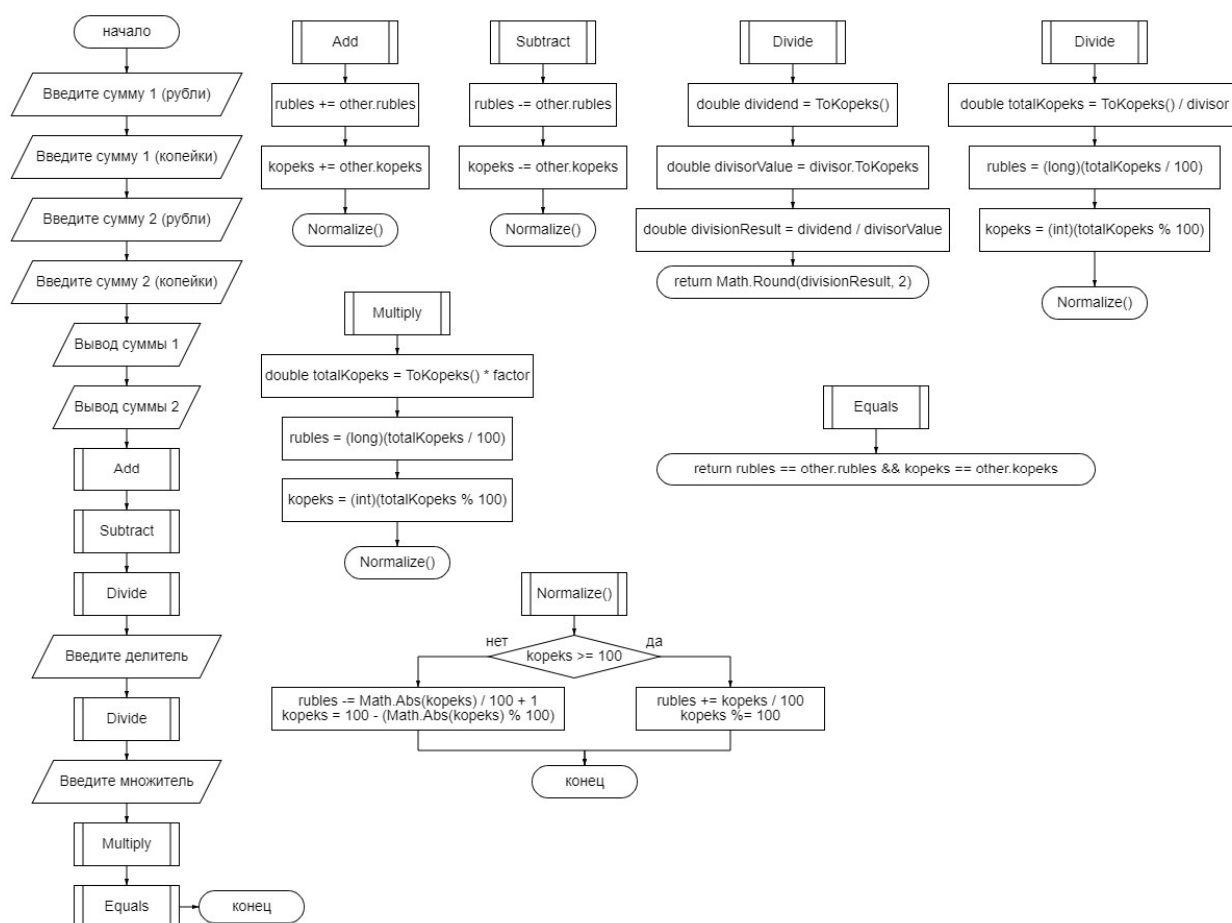


Рисунок 2.1 – Блок-схема к задаче

Реализация в виде программного кода.

```
using System;

namespace Лаба_6._1
{
    interface IMoney
    {
        void Add(Money other);
        void Subtract(Money other);
        double Divide(Money divisor);
        void Divide(double divisor);
        void Multiply(double factor);
        bool Equals(Money other);
    }

    abstract class Money : IMoney
    {
        protected long rubles;
        protected int kopeks;

        public Money(long rubles, int kopeks)
        {
            this.rubles = rubles;
            this.kopeks = kopeks;
        }

        public void Add(Money other)
        {
            rubles += other.rubles;
            kopeks += other.kopeks;

            Normalize();
        }

        public void Subtract(Money other)
        {
            rubles -= other.rubles;
            kopeks -= other.kopeks;

            Normalize();
        }

        public double Divide(Money divisor)
        {
            double dividend = ToKopeks();
            double divisorValue = divisor.ToKopeks();

            double divisionResult = dividend / divisorValue;

            return Math.Round(divisionResult, 2);
        }

        public void Divide(double divisor)
        {
            double totalKopeks = ToKopeks() / divisor;

            rubles = (long)(totalKopeks / 100);
            kopeks = (int)(totalKopeks % 100);

            Normalize();
        }

        public void Multiply(double factor)
        {

```

```

        double totalKopeks = ToKopeks() * factor;

        rubles = (long)(totalKopeks / 100);
        kopeks = (int)(totalKopeks % 100);

        Normalize();
    }

    public bool Equals(Money other)
    {
        return rubles == other.rubles && kopeks == other.kopeks;
    }

    public override string ToString()
    {
        string result = $"{rubles},{kopeks:00}";
        return result;
    }

    protected double ToKopeks()
    {
        return rubles * 100 + kopeks;
    }

    protected void Normalize()
    {
        if (kopeks >= 100)
        {
            rubles += kopeks / 100;
            kopeks %= 100;
        }
        else if (kopeks < 0)
        {
            rubles -= Math.Abs(kopeks) / 100 + 1;
            kopeks = 100 - (Math.Abs(kopeks) % 100);
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.Write("Введите сумму 1 (рубли): ");
        long rubles1 = long.Parse(Console.ReadLine());

        Console.Write("Введите сумму 1 (копейки): ");
        int kopeks1 = int.Parse(Console.ReadLine());

        Console.Write("Введите сумму 2 (рубли): ");
        long rubles2 = long.Parse(Console.ReadLine());

        Console.Write("Введите сумму 2 (копейки): ");
        int kopeks2 = int.Parse(Console.ReadLine());

        Money money1 = new MoneyImpl(rubles1, kopeks1);
        Money money2 = new MoneyImpl(rubles2, kopeks2);

        Console.WriteLine($"Сумма 1: {money1} руб.");
        Console.WriteLine($"Сумма 2: {money2} руб.");

        money1.Add(money2);
        Console.WriteLine($"Сумма 1 + Сумма 2 = {money1} руб.");

        money1 = new MoneyImpl(rubles1, kopeks1);
    }
}

```

```

money1.Subtract(money2);
Console.WriteLine($"Сумма 1 - Сумма 2 = {money1} руб.");

money1 = new MoneyImpl(rubles1, kopeks1);
double divisionResult = money1.Divide(money2);
Console.WriteLine($"Сумма 1 / Сумма 2 = {divisionResult}");

money1 = new MoneyImpl(rubles1, kopeks1);
Console.Write("Введите делитель: ");
double divisor = double.Parse(Console.ReadLine());
money1.Divide(divisor);
Console.WriteLine($"Сумма 1 / {divisor} = {money1} руб.");

money1 = new MoneyImpl(rubles1, kopeks1);
Console.Write("Введите множитель: ");
double factor = double.Parse(Console.ReadLine());
money1.Multiply(factor);
Console.WriteLine($"Сумма 1 * {factor} = {money1} руб.");

money1 = new MoneyImpl(rubles1, kopeks1);
Console.WriteLine($"Сумма 1 == Сумма 2? {money1.Equals(money2)}");
    }
}

class MoneyImpl : Money
{
    public MoneyImpl(long rubles, int kopeks) : base(rubles, kopeks)
    {
    }
}
}

```

Результат работы программного кода представлен на рисунке 2.2.

```

C:\WINDOWS\system32\cmd.exe
Введите сумму 1 (рубли): 1571
Введите сумму 1 (копейки): 0
Введите сумму 2 (рубли): 1689
Введите сумму 2 (копейки): 50
Сумма 1: 1571,00 руб.
Сумма 2: 1689,50 руб.
Сумма 1 + Сумма 2 = 3260,50 руб.
Сумма 1 - Сумма 2 = -119,50 руб.
Сумма 1 / Сумма 2 = 0,93
Введите делитель: 0,5
Сумма 1 / 0,5 = 3142,00 руб.
Введите множитель: 2,5
Сумма 1 * 2,5 = 3927,50 руб.
Сумма 1 == Сумма 2? False
Для продолжения нажмите любую клавишу . . .

```

Рисунок 2.2 – Результат работы программы

2.2 Классы. Задание 2

Задание вариант 4: Создать класс Goods (товар). В классе должны быть представлены поля: наименование товара, дата оформления, цена товара, количество единиц товара, номер накладной, по которой товар поступил на склад. Реализовать методы изменения цены товара, изменения количества товара (увеличения и уменьшения), вычисления стоимости товара. Должен быть метод для отображения стоимости товара в виде строки.

Работа выполнялась в паре с Вологодским Егором гр. 722-1.

Графический способ реализации данного алгоритма в виде блок-схемы представлен на рисунке 2.3.

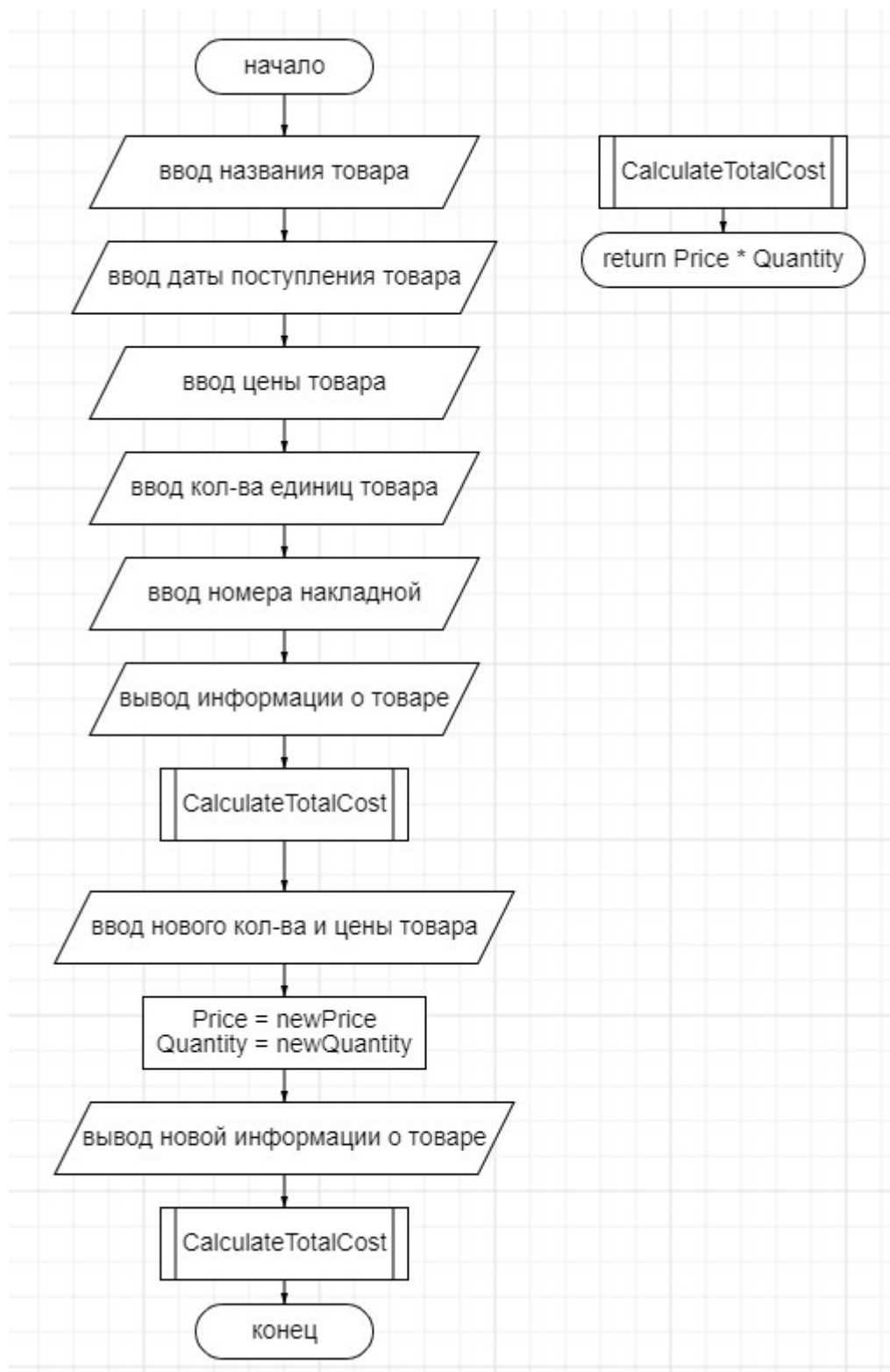


Рисунок 2.3 – Блок-схема к задаче

Реализация в виде программного кода.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```



```

namespace Лаба_6._2
{
    public abstract class Goods
    {
        protected string productName;
        protected DateTime registrationDate;
        protected decimal price;
        protected int quantity;
        protected string invoiceNumber;

        public string ProductName
        {
            get { return productName; }
            set { productName = value; }
        }

        public DateTime RegistrationDate
        {
            get { return registrationDate; }
            set { registrationDate = value; }
        }

        public decimal Price
        {
            get { return price; }
            set { price = value; }
        }

        public int Quantity
        {
            get { return quantity; }
            set { quantity = value; }
        }

        public string InvoiceNumber
        {
            get { return invoiceNumber; }
            set { invoiceNumber = value; }
        }

        public Goods(string productName, DateTime registrationDate, decimal price,
int quantity, string invoiceNumber)
        {
            this.productName = productName;
            this.registrationDate = registrationDate;
            this.price = price;
            this.quantity = quantity;
            this.invoiceNumber = invoiceNumber;
        }

        public abstract void ChangePrice(decimal newPrice);
        public abstract void ChangeQuantity(int newQuantity);
        public abstract decimal CalculateTotalCost();
        public abstract string GetCostAsString();
    }

    public interface IPriceChangeable
    {
        void ChangePrice(decimal newPrice);
    }

    public interface IQuantityChangeable
    {
        void ChangeQuantity(int newQuantity);
    }
}

```

```

    }

    public class ConcreteGoods : Goods, IPriceChangeable, IQuantityChangeable
    {
        public ConcreteGoods(string productName, DateTime registrationDate, decimal
price, int quantity, string invoiceNumber)
            : base(productName, registrationDate, price, quantity, invoiceNumber)
        {
        }

        public override void ChangePrice(decimal newPrice)
        {
            Price = newPrice;
        }

        public override void ChangeQuantity(int newQuantity)
        {
            Quantity = newQuantity;
        }

        public override decimal CalculateTotalCost()
        {
            return Price * Quantity;
        }

        public override string GetCostAsString()
        {
            return CalculateTotalCost().ToString();
        }
    }

    public class Program
    {
        public static void Main(string[] args)
        {
            Console.Write("Введите наименование товара: ");
            string productName = Console.ReadLine();

            Console.Write("Введите дату оформления товара (гггг-мм-дд): ");
            DateTime registrationDate = DateTime.Parse(Console.ReadLine());

            Console.Write("Введите цену товара: ");
            decimal price = decimal.Parse(Console.ReadLine());

            Console.Write("Введите количество единиц товара: ");
            int quantity = int.Parse(Console.ReadLine());

            Console.Write("Введите номер накладной: ");
            string invoiceNumber = Console.ReadLine();

            Goods goods = new ConcreteGoods(productName, registrationDate, price,
quantity, invoiceNumber);

            Console.WriteLine("\nИнформация о товаре:");
            Console.WriteLine("Наименование: " + goods.ProductName);
            Console.WriteLine("Дата оформления: " +
goods.RegistrationDate.ToShortDateString());
            Console.WriteLine("Цена: " + goods.Price.ToString() + " руб.");
            Console.WriteLine("Количество: " + goods.Quantity);
            Console.WriteLine("Номер накладной: " + goods.InvoiceNumber);
            Console.WriteLine("Общая стоимость: " + goods.GetCostAsString() + "
руб.");

            Console.WriteLine("\nИзменение цены товара.");
            Console.Write("Введите новую цену товара: ");

```

```

decimal newPrice = decimal.Parse(Console.ReadLine());
((IPriceChangeable)goods).ChangePrice(newPrice);

Console.WriteLine("\nИзменение количества товара.");
Console.Write("Введите новое количество товара: ");
int newQuantity = int.Parse(Console.ReadLine());
((IQuantityChangeable)goods).ChangeQuantity(newQuantity);

Console.WriteLine("\nИнформация о товаре после изменений:");
Console.WriteLine("Цена: " + goods.Price.ToString() + " руб.");
Console.WriteLine("Количество: " + goods.Quantity);
Console.WriteLine("Общая стоимость: " + goods.GetCostAsString() + "
руб.");
    }
}
}

```

Результат работы программного кода представлен на рисунке 2.4.

```

C:\WINDOWS\system32\cmd.exe
Введите наименование товара: сыр
Введите дату оформления товара (гггг-мм-дд): 2003 11 27
Введите цену товара: 150
Введите количество единиц товара: 10
Введите номер накладной: 256

Информация о товаре:
Наименование: сыр
Дата оформления: 27.11.2003
Цена: 150 руб.
Количество: 10
Номер накладной: 256
Общая стоимость: 1500 руб.

Изменение цены товара.
Введите новую цену товара: 160

Изменение количества товара.
Введите новое количество товара: 20

Информация о товаре после изменений:
Цена: 160 руб.
Количество: 20
Общая стоимость: 3200 руб.
Для продолжения нажмите любую клавишу . . .

```

Рисунок 2.4 – Результат работы программы

2.13 Полиморфизм

Задание вариант 4: Описать классы «Рыба», «Животное», «Млекопитающие», «Птица», «Пингвин», «Дельфин». Выстроить правильную иерархию наследования этих классов, определить какие из классов должны быть абстрактными, какие методы абстрактными или виртуальными. Описать конструкторы и деструкторы разработанных классов. Создать по одному объекту каждого класса, для которого это возможно, поместить их в один

массив. Для каждого объекта вызвать метод Move (способ передвижения) и вывести на экран все свойства каждого из объектов.

Схема программы представленная на UML-диаграмме на рисунке 2.5.

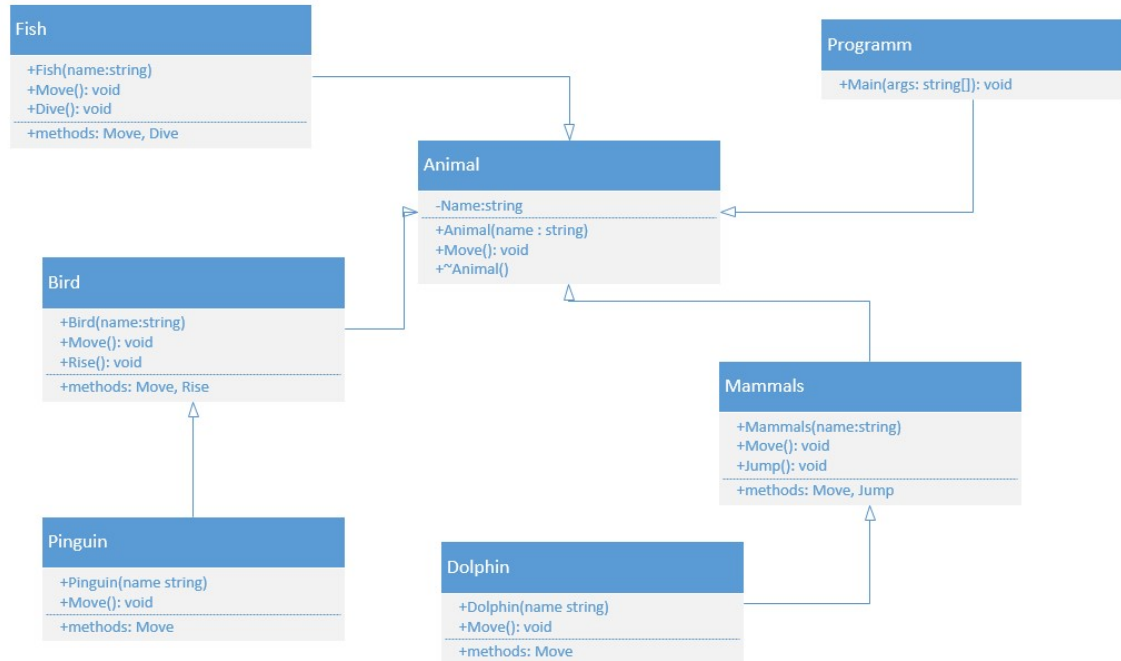


Рисунок 2.5 – UML-диаграмма

Реализация в виде программного кода.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Лаба_7
{
    // Абстрактный класс Животное
    abstract class Animal
    {
        public string Name { get; set; }

        public abstract void Move();

        // Конструктор класса Животное
        public Animal(string name)
        {
            Name = name;
        }

        // Деструктор класса Животное
        ~Animal()
        {
            Console.WriteLine($"Объект {Name} класса Animal уничтожен.");
        }
    }
}

```

```

    }
}

// Абстрактный Класс Рыба, наследуется от Животное
abstract class Fish : Animal
{
    public Fish(string name) : base(name) { }

    public abstract void Dive();
}

// Абстрактный Класс Птица, наследуется от Животное
abstract class Bird : Animal
{
    public Bird(string name) : base(name) { }

    public abstract void Rise();
}

// Класс Дельфин, наследуется от Животное
class Dolphin : Animal
{
    public Dolphin(string name) : base(name) { }

    public override void Move()
    {
        Console.WriteLine($"{Name} says: I'm swimming");
    }
}

// Класс Пингвин, наследуется от Животное
class Penguin : Animal
{
    public Penguin(string name) : base(name) { }

    public override void Move()
    {
        Console.WriteLine($"{Name} says: I'm swimming and walking");
    }
}

abstract class Mammals : Animal
{
    public Mammals(string name) : base(name) { }

    public abstract void Jump();
}

// Пример класса, реализующего интерфейс IDisposable, для демонстрации
// деструктора
class ExampleDisposable : IDisposable
{
    public ExampleDisposable()
    {
        Console.WriteLine("Создан объект ExampleDisposable.");
        Console.ReadLine();
    }

    public void Dispose()
    {
        Console.ReadLine();
        Console.WriteLine("Вызван метод Dispose объекта ExampleDisposable.");
    }
}

class Program

```

```

{
    static void Main(string[] args)
    {
        Animal[] animals = new Animal[3];

        animals[0] = new Dolphin("Теранс");
        animals[1] = new Penguin("Ковальски");

        // Создание объекта ExampleDisposable для демонстрации деструктора
        using (var exampleDisposable = new ExampleDisposable())
        {
            animals[2] = new Penguin("Шкипер");
            foreach (var animal in animals)
            {
                animal.Move();
            }
        }

        Console.ReadLine();
    }
}

```

Результат работы программного кода представлен на рисунке 2.6.

```

C:\WINDOWS\system32\cmd.exe
Создан объект ExampleDisposable.
Теранс says: I'm swimming
Ковальски says: I'm swimming and walking
Шкипер says: I'm swimming and walking
Вызван метод Dispose объекта ExampleDisposable.
Объект Шкипер класса Animal уничтожен.
Объект Ковальски класса Animal уничтожен.
Объект Теранс класса Animal уничтожен.
Для продолжения нажмите любую клавишу . . .

```

Рисунок 2.6 – Результат работы программы

Заключение

В ходе выполнения лабораторных работ были получены навыки построения алгоритмов, а также их написание на языке программирования C# на компьютере. Также были получены навыки по решению различных задач по обработке информации и моделированию.